CharAnalysis

Modernization & R-Translation Report

Action Plan for Code Efficiency and Language Portability

Executive Summary

CharAnalysis is a widely-used MATLAB program for reconstructing local fire histories from lake-sediment charcoal records. Originally developed in the mid-2000s, the codebase reflects the conventions and toolbox constraints of MATLAB 7.x. This report identifies specific improvements to modernize the code, increase computational efficiency, and establish a clear pathway for translating the program into R — the dominant language in the paleoecological community today.

Three categories of work are recommended:

- MATLAB modernization — eliminating legacy patterns, vectorizing inner loops, and removing deprecated function calls.
- Architecture improvements — separating computation from visualization, introducing a formal parameter object, and adding input validation.
- R translation strategy — a module-by-module roadmap with notes on where native MATLAB features require alternative implementations in R.

1. Codebase Overview

1.1 Program Architecture

CharAnalysis is organized as a set of 23 MATLAB .m function files orchestrated by a single top-level entry point (CharAnalysis.m). The call hierarchy is as follows:

```
CharAnalysis.m  (entry point)
  ├── CharParameters.m          Read .xls / .csv input file
  ├── CharPretreatment.m         Interpolate, compute CHAR, transform
  ├── CharSmooth.m               Estimate Cbackground (5 methods)
  ├── CharThreshGlobal.m         Global threshold + noise PDF
  ├── CharThreshLocal.m          Local (sliding-window) threshold
  ├── CharPeakID.m               Peak flagging + minimum-count screen
  ├── CharPeakAnalysisResults.m  All 9-10 output figures + CSV/XLS save
  └── bkgCharSensitivity.m       Sensitivity to Cbackground window width
```

The Gaussian Mixture Model sub-system (GaussianMixture.m, EStep.m, MStep.m, EMIterate.m, MDLReduceOrder.m, ClusterNormalize.m, initMixture.m, SplitClasses.m, GMClassLikelihood.m) is a self-contained EM implementation bundled to free users from requiring the Statistics Toolbox.

1.2 Key Data Structures

Data flow through the program via three primary structs and several scalar/vector parameter structs:

Struct / Variable	Contents
Charcoal	Raw and resampled time series: depths, ages, counts, volumes, concentrations, CHAR, smoothed CHAR, peak CHAR, identified peaks, magnitudes, fire frequencies
CharThresh	Threshold values (pos/neg, 4 columns), noise PDF, SNI, GOF, minCountP
Pretreatment	zoneDiv, yrInterp, transform
Smoothing	method (1–5), yr (window width)
PeakAnalysis	cPeak, threshType, threshMethod, threshValues[4], minCountP, peakFrequ, bkgSens
Results	save, saveFigures, allFigures

2. Efficiency Improvements

2.1 Vectorize Inner Loops

The most computationally expensive bottleneck is the nested loop in CharPretreatment.m that builds the proportion matrix used for weighted interpolation. For a typical record of ~500 raw samples resampled to ~1000 intervals, this loop executes up to 500,000 iterations:

```
for i = 1:length(Charcoal.ybpI)      % ~1000 iterations
  for j = 1:length(Charcoal.ybp)     % ~500 iterations
    % 4 conditional branches to compute propMatrix(i,j)
  end
end
```

This can be fully vectorized using MATLAB broadcasting (or bsxfun for older targets):

```
% Vectorized proportion matrix — no inner loop
rsAgeTop = Charcoal.ybpI;            % column vector [N_rs x 1]
rsAgeBot = rsAgeTop + yrInterp;
aT = ageTop';  aB = ageBot';         % row vectors  [1 x N_raw]
% Each of the four cases becomes a logical mask:
caseA = (aT >= rsAgeTop) & (aT < rsAgeBot) & (aB > rsAgeBot);
caseB = (aT < rsAgeTop) & (aB <= rsAgeBot) & (aB > rsAgeTop);
caseC = (aT >= rsAgeTop) & (aB <= rsAgeBot);
caseD = (aT < rsAgeTop) & (aB > rsAgeBot);
propMatrix = caseA.*(rsAgeBot - aT) + caseB.*(aB - rsAgeTop) ...
           + caseC.*(aB - aT) + caseD.*yrInterp;
propMatrix = propMatrix ./ yrInterp;
```

WARNING Benchmark this change with the existing test datasets before replacing the production code. Floating-point accumulation order differs slightly between loops and vectorized sums; results should be identical to within numerical tolerance (~1e-12).

2.2 Vectorize the Peak-Flagging Loop in CharPeakID.m

The peak identification loop iterates over every sample for every threshold, an O(N * T) double loop where N can be thousands and T is 4. The inner logic is simple comparison and state-tracking:

```
for i = 1:length(Charcoal.peak)
  for j = 1:nThresholds
    if Charcoal.peak(i) > thresholdValues(i,j)
      Charcoal.charPeaks(i,j) = 2;
    end
  end
end
```

This reduces to a single comparison and a logical-to-numeric cast:

```
Charcoal.charPeaks = 2 * (Charcoal.peak > thresholdValues);
```

The subsequent consecutive-peak removal loop (which walks forward through the array) cannot be trivially vectorized but can be replaced with a diff-based approach:

```
for j = 1:nThresholds
  pk = Charcoal.charPeaks(:,j);
  transitions = [pk(1)>0; diff(pk>0) < 0];  % falling edge = start of run
  Charcoal.charPeaks(:,j) = (pk == 2) & transitions;
end
```

2.3 Replace the CharThreshLocal.m Sample-by-Sample Loop

CharThreshLocal.m iterates over every interpolated sample (~1000+) to fit a noise distribution within a sliding window. This is the single slowest function in the program. Two improvements are possible:

1. Pre-compute the window indices once before the loop using a logical index matrix, avoiding the three-branch if/else inside the loop.
2. For threshMethod == 2 (Gaussian noise), compute sigma for all windows simultaneously using cumulative sums (O(N) instead of O(N * W/r)):

```
% Running sigma of values <= 0 using cumsum trick
negMask = double(Charcoal.peak <= 0);
mirrorPeak = [Charcoal.peak(Charcoal.peak<=0); -Charcoal.peak(Charcoal.peak<0)];
% Then compute running std over mirrored series via movstd (R2016a+)
sigmaHat = movstd(mirrorPeak, round(threshYr/r));
```

2.4 Preallocate All Arrays

Several functions grow arrays inside loops using concatenation, which forces repeated memory reallocation. Specifically:

- In smoothFRI.m, FRImatrix.FRI and FRImatrix.params are preallocated correctly, but p, p2, and p3 arrays are built via index assignment without preallocation — the correct pre-allocation already appears in comments but is incomplete.
- In CharPeakAnalysisResults.m (Figure 7 section), peak_in and peak_mag are preallocated with zeros, which is correct, but the loop could be replaced with a vectorized run-length encoding approach.

2.5 Remove Global Variables

The program uses two global variables (plotData, bkgSensIn) as state flags shared across functions. Global state is a common source of hard-to-debug side effects. Both should be converted to function arguments:

```
% Current pattern (fragile):
global plotData
global bkgSensIn

% Recommended: pass as explicit arguments
function [Charcoal] = CharSmooth(Charcoal, Pretreatment, Smoothing, Results, plotData)
```

NOTE The bkgSensIn global is especially fragile: it is declared, cleared, and re-declared across CharParameters.m and bkgCharSensitivity.m. This can cause incorrect threshold-bin calculations if functions are called out of expected order.

2.6 Eliminate Deprecated Functions

Several functions used in CharAnalysis have been deprecated in modern MATLAB releases:

Deprecated Call	Modern Replacement	File(s) Affected
plotyy()	yyaxis (R2016a+)	CharPeakAnalysisResults.m (×3)
[AX,H1,H2]=plotyy()	yyaxis left/right	CharPeakAnalysisResults.m
bkgCharSensitivity_conPDF.m (duplicate)	Merge into bkgCharSensitivity.m	bkgCharSensitivity_conPDF.m
[h,p,t]=kstest(...) 3-output form	Use [h,p]=kstest(...)	CharPeakAnalysisResults.m, CharThreshLocal.m
xlswrite() / xlsread()	writematrix() / readmatrix()	CharParameters.m, CharPeakAnalysisResults.m
smooth() (Curve Fitting Toolbox)	movmean, movmedian, or custom	CharSmooth.m, CharThreshLocal.m, smoothFRI.m
hist() (use histogram())	histcounts() for numeric output	CharPeakAnalysisResults.m, CharThreshLocal.m
[find(x == max(x))]	max(x,'linear') or find+max	CharThreshLocal.m

WARNING The smooth() function from the Curve Fitting Toolbox is called in at least 8 places across 4 files. If the toolbox is unavailable, the program silently fails. Replacing these calls with base-MATLAB equivalents (movmean, movmedian, a custom lowess) is the single most important compatibility fix.

2.7 Duplicate File

The file bkgCharSensitivity_conPDF.m is an older version of bkgCharSensitivity.m that remains in the distribution. It defines a function with the same name, which will cause a name collision depending on MATLAB path ordering. This file should be removed or clearly archived.

3. Architecture Improvements

3.1 Separate Computation from Visualization

CharPeakAnalysisResults.m is 800+ lines and combines all analytical post-processing with all figure generation. This makes unit testing impossible and translation to R extremely difficult. The recommended split is:

```
CharPeakAnalysisResults.m  →  CharPostProcess.m   (pure computation)
                           ↳  CharPlotResults.m   (all figure calls)
```

CharPostProcess.m would compute: peak magnitudes, smoothed FRIs, fire frequency time series, FRI distributions by zone, Weibull fits, bootstrapped CIs, and KS test results — returning all results as fields of Charcoal and summary structs. CharPlotResults.m would only call plotting functions using those pre-computed values.

3.2 Input Validation Function

Currently, parameter validation occurs inline in CharAnalysis.m with a small block of if/error statements. This should be expanded into a dedicated CharValidateParams.m that checks:

- Smoothing window shorter than record length
- threshMethod/threshType combination validity
- threshValues all in (0,1) when threshMethod > 1
- yrInterp > 0 or == 0 (auto)
- zoneDiv in ascending order with at least 2 values
- charData column count and no NaN ages

3.3 Structured Parameter Object

Rather than passing 5–6 separate structs to every function, consolidate into a single params struct with named sub-fields. This is already partially done (Pretreatment, Smoothing, PeakAnalysis, Results) but each function has slightly different calling conventions. A unified interface would be:

```
params.pretreatment  = Pretreatment;
params.smoothing     = Smoothing;
params.peakAnalysis  = PeakAnalysis;
```

```
params.results        = Results;
params.site           = site;
```

3.4 Unit Tests

No test infrastructure currently exists. At minimum, a test script should be created that runs CharAnalysis on the bundled template dataset and asserts that key output values match known-good reference outputs. MATLAB's built-in testing framework (matlab.unittest) can be used for this, or a simple assert-based script suffices.

4. R Translation Strategy

4.1 Overview

R is now the dominant language in the paleoecological community. An R implementation of CharAnalysis would significantly broaden accessibility. The table below provides a high-level module-by-module translation map:

MATLAB Module	R Equivalent	Difficulty	Notes
CharParameters.m	readxl::read_excel() or read.csv()	Low	Well-supported; xls/csv both handled natively in R
CharPretreatment.m	Pure base R	Low	Proportion-matrix interpolation maps directly to matrix operations
CharSmooth.m — lowess/rlowess	stats::lowess() or loess()	Low-Med	R lowess() slightly different span definition; needs calibration
CharSmooth.m — moving avg	zoo::rollmean() or base filter()	Low	Direct equivalent
CharSmooth.m — moving median	zoo::rollmedian()	Low	Direct equivalent
CharSmooth.m — moving mode	Custom R function	Medium	No base-R equivalent; implement with lapply + tabulate
CharThreshGlobal.m	stats::qnorm(), stats::dnorm()	Low	Gaussian threshold trivially equivalent
CharThreshLocal.m	stats::qnorm() + rolling window	Medium	Sliding window in R with zoo or a custom lapply loop
CharPeakID.m	Base R vectorized	Low	All logic is comparison + diff; maps cleanly
CharPostProcess (new)	Base R + survival package	Low-Med	Weibull: survival::survreg() or MASS::fitdistr()
GaussianMixture (EM cluster)	mclust::Mclust(G=2)	Low	mclust is mature, well-tested, and widely used
smoothFRI.m	stats::loess() + bootstrap	Medium	Anderson-Darling test: kSamples::ad.test()

outputResults.m	write.csv() or openxlsx	Low	Trivial
CharPeakAnalysisResults (plots)	ggplot2	High	See Section 4.3

4.2 Recommended R Package Dependencies

A well-structured R translation would rely on the following packages:

Package	Purpose
readxl / openxlsx	Read and write Excel files (replaces xlsread/xlswrite)
zoo	Rolling statistics: rollmean, rollmedian, rollapply
mclust	Gaussian mixture models (replaces the bundled EM implementation)
ggplot2	All output figures
patchwork	Multi-panel figure assembly (replaces MATLAB subplot layout)
MASS	fitdistr() for Weibull parameter estimation
survival	survreg() as alternative Weibull fitter for FRI data
kSamples	ad.test() for Anderson-Darling test (used in smoothFRI.m)
boot	Bootstrap confidence intervals
stats (base)	ks.test(), ks.test(two-sample), qnorm, pnorm, dnorm, lowess

4.3 Visualization: High-Effort but Achievable

The output figures in CharPeakAnalysisResults.m are the most complex part of the translation. Each of the 9–10 figures uses MATLAB-specific layout commands (subplot, plotyy, dual y-axes, arbitrary position vectors). In R with ggplot2 and patchwork, the equivalent logic is available but requires a near-complete rewrite of each figure.

The recommended approach is to treat figure output as a separate phase — translate the computation first into R, validate output numerically against MATLAB results, and then build the ggplot2 figures iteratively:

3. Phase 1: Core computation only (CharPretreatment, CharSmooth, CharThreshGlobal/Local, CharPeakID). Validate numerically.
4. Phase 2: Post-processing outputs (FRI distributions, Weibull fits, SNI, GOF, peak magnitudes). Validate numerically.
5. Phase 3: ggplot2 figure equivalents, one figure at a time.
6. Phase 4: Input/output wrappers, documentation, CRAN submission if desired.

5. Translation Issues and Blockers

5.1 smooth() — Curve Fitting Toolbox Dependency

The MATLAB smooth() function (Curve Fitting Toolbox) is called 9 times across 4 files. The R equivalent is stats::lowess() for the lowess variant, but there are critical differences:

- MATLAB smooth(x, span, 'lowess') uses span as a fraction of data length if < 1, or number of points if >= 1.
- R lowess(x, f) uses f as a fraction of data points, always in (0,1).
- For the 'rlowess' (robust lowess) variant, R lowess() also applies robustness iterations — the default iter=3 is typically a close match.

NOTE A thin wrapper function in R that maps MATLAB's span convention to R's f convention will simplify the translation and reduce per-call errors. Test on 3–4 reference datasets before finalizing.

5.2 Weibull Fitting — wblfit() and wblpdf()

MATLAB's wblfit() fits a Weibull distribution to frequency data (histogram bins + counts) using weighted maximum likelihood. R's MASS::fitdistr() fits to raw data, not to pre-binned data. For FRI analysis, the appropriate R translation depends on whether raw FRI values are available:

- If raw FRIs are available (they are, as diff(peakYr)): use MASS::fitdistr(FRI, 'weibull') directly. This is simpler and more statistically sound than the histogram-based fitting in MATLAB.
- The MATLAB wblcdf() / wblpdf() functions map directly to R's pweibull() / dweibull().
- Parameterization note: MATLAB uses (a = scale, b = shape); R uses (shape, scale) in that order. Swap when translating.

WARNING MATLAB's Weibull shape parameter is labeled 'b' in the output display; R labels it 'shape'. These are the same quantity. The scale parameter MATLAB calls 'a' is R's 'scale'. Double-check parameter order in all downstream calculations.

5.3 kstest() — Single-Sample vs. Two-Sample

CharAnalysis uses MATLAB's kstest() in the single-sample form (test against a specified CDF) to evaluate goodness-of-fit of FRI data to a Weibull model. R's ks.test() in its default form tests for normality or against a specified distribution:

```
# R equivalent of MATLAB: [h,p] = kstest(FRI, [bins', wbl_cdf'])
# Use ks.test with the theoretical CDF:
p_ks <- ks.test(FRI, 'pweibull', shape=param[2], scale=param[1])$p.value
```

5.4 Anderson-Darling Test — AnDarksamtest()

CharAnalysis calls AnDarksamtest() in smoothFRI.m for between-window FRI comparisons. This is not a base-MATLAB function — it was a custom file bundled with the distribution. In R, the kSamples package provides ad.test() as a direct equivalent. The calling convention differs:

```
# MATLAB (custom): p = AnDarksamtest(x12, alpha)  where x12 is [values, groupID]
# R equivalent:
library(kSamples)
result <- ad.test(list(x1, x2), method='asymptotic')
p_value <- result$ad[1, 'P-value']
```

5.5 Gaussian Mixture Model — Custom EM vs. mclust

The bundled EM implementation (GaussianMixture.m and its 8 supporting files) implements a Gaussian mixture model with MDL-based order selection. In R, the mclust package provides a mature, tested equivalent:

```
library(mclust)
fit <- Mclust(X, G=2, modelNames='E')  # equal variance, 2 components
mu   <- fit$parameters$mean
sig  <- sqrt(fit$parameters$variance$sigmasq)
prop <- fit$parameters$pro
```

One important difference: mclust uses BIC for model selection; the MATLAB implementation uses Rissanen's MDL criterion. For G=2 forced (as used in CharAnalysis), this difference does not apply. However, for the auto-order case (finalK=0), BIC and MDL will occasionally disagree on the optimal number of components. Testing on several real charcoal records is recommended before finalizing this substitution.

5.6 bootstrp() — Bootstrap Function

MATLAB's bootstrp() is called 6 times across CharPeakAnalysisResults.m and CharThreshLocal.m. The R equivalent is boot::boot():

```
# MATLAB: boot_means = bootstrp(1000, 'mean', FRI)
# R:
library(boot)
boot_means <- boot(FRI, function(x,i) mean(x[i]), R=1000)$t
ci <- quantile(boot_means, c(0.025, 0.975))
```

5.7 tcdf() — Student's T CDF (Minimum Count Analysis)

CharPeakID.m uses tcdf(d, 10^10) to compute a p-value for the minimum-count test. With 10^10 degrees of freedom, the t-distribution is numerically identical to the standard normal. In R, this is simply:

```
# MATLAB: CharThresh.minCountP = 1 - tcdf(d, 1e10)
# R (using normal approximation, which is exact at df = 1e10):
min_count_p <- 1 - pnorm(d)
```

5.8 Figure Layout — subplot() and Position Vectors

MATLAB's figure layout relies on subplot(), manually specified Position vectors (normalized figure coordinates), and the plotyy() dual-axis function. None of these have direct R equivalents. The recommended approach:

- Use ggplot2 for individual panels.
- Use patchwork for multi-panel layouts (replaces subplot).
- Use sec_axis() in ggplot2 for dual y-axes (replaces plotyy).
- Accept that figures will look different from MATLAB output — focus on information content, not pixel-level reproduction.

6. Prioritized Action Plan

Phase 1 — MATLAB Modernization (Recommended Before R Translation)

These changes make the MATLAB code cleaner, reduce toolbox dependencies, and produce a stable tested baseline for comparison during R translation. Estimated effort: 2–4 weeks.

Task	File(s)	Priority	Est. Hours
Remove bkgCharSensitivity_conPDF.m (duplicate)	bkgCharSensitivity_conPDF.m	High	0.5
Replace plotyy() with yyaxis	CharPeakAnalysisResults.m	High	4
Replace xlsread/xlswrite with readmatrix/writematrix	CharParameters.m, CharPeakAnalysisResults.m	High	2
Replace hist() with histcounts()	CharPeakAnalysisResults.m, CharThreshLocal.m	Medium	2
Remove global variables (plotData, bkgSensIn)	All files	High	4
Vectorize propMatrix loop (CharPretreatment.m)	CharPretreatment.m	High	3
Vectorize charPeaks assignment (CharPeakID.m)	CharPeakID.m	Medium	2
Split CharPeakAnalysisResults into compute + plot	CharPeakAnalysisResults.m	High	8
Add CharValidateParams.m	New file	Medium	4
Replace smooth() calls with base-MATLAB equivalents	CharSmooth.m, CharThreshLocal.m, smoothFRI.m	High	8
Add regression test script (known-good outputs)	New file	High	4

Phase 2 — R Translation Core (Computation Only)

Translate all analytical functions to R without any figures. Validate numerically against Phase 1 MATLAB outputs. Estimated effort: 4–6 weeks.

7. Implement char_parameters() reading both CSV and Excel input formats.
8. Implement char_pretreatment() with vectorized proportion-matrix interpolation.
9. Implement char_smooth() with zoo rolling functions and a custom lowess wrapper.
10. Implement char_thresh_global() and char_thresh_local() using stats::qnorm/pnorm.
11. Replace the bundled EM implementation with mclust::Mclust(G=2).
12. Implement char_peak_id() using vectorized comparison and diff().

13. Implement output_results() using write.csv() or openxlsx.
14. Create numerical validation script comparing R output to MATLAB output on test datasets.

Phase 3 — R Translation Figures and Packaging

Build ggplot2 figure equivalents and package for distribution. Estimated effort: 4–8 weeks.

15. Build ggplot2 equivalents for all 9 standard output figures.
16. Build Figure 10 (sensitivity) using patchwork for layout.
17. Write roxygen2 documentation for all public functions.
18. Create a vignette demonstrating the full workflow on the bundled template data.
19. Consider CRAN submission or distribution via GitHub with remotes::install_github().

7. Translation Risk Summary

The following table summarizes the risk and effort for each identified translation issue:

Issue	Risk Level	R Alternative	Validation Required?
smooth() lowess span convention	Medium	stats::lowess() with wrapper	Yes — compare on 3+ datasets
Weibull parameterization order	High	MASS::fitdistr(); swap a/b	Yes — check all Weibull outputs
kstest() single-sample form	Low	ks.test() with pweibull()	Yes
AnDarksamtest() custom function	Low	kSamples::ad.test()	Yes
mclust vs. MDL order selection	Medium	mclust::Mclust(G=2, fixed)	Yes — compare mu/sig on 5+ records
bootstrp() convention	Low	boot::boot()	Spot-check
tcdf(d, 1e10) approximation	Very Low	pnorm(d)	No — mathematically equivalent
subplot/Position figure layout	High	ggplot2 + patchwork	Manual inspection only
plotyy dual y-axis	Medium	ggplot2 sec_axis()	Manual inspection
Global variables (plotData, bkgSensIn)	High (bug risk)	Function arguments	Regression test
bkgCharSensitivity duplicate file	High (name collision)	Delete the file	No

8. Recommended Resources

- mclust package: Scrucca et al. (2016) mclust 5: clustering, classification and density estimation using Gaussian finite mixture models. The R Journal 8(1), 289-317.
- zoo package: Zeileis & Grothendieck (2005) zoo: S3 Infrastructure for Regular and Irregular Time Series. Journal of Statistical Software 14(6).
- kSamples package: Scholz & Zhu (2019) kSamples: K-Sample Rank Tests and their Combinations. CRAN.
- patchwork package: Pedersen (2020) patchwork: The Composer of Plots. CRAN.
- CharAnalysis GitHub: https://github.com/phiguera/CharAnalysis
- Original methods paper: Higuera et al. (2009) Vegetation mediated the impacts of postglacial climate change on fire regimes in the south-central Brooks Range, Alaska. Ecological Monographs 79(2): 201-219.

— End of Report —